

COMP 560 Final Report: Object Newness Perception in the Open World

Thomas You, Yuyang Deng, Junyi Zhang

Abstract

We address open-world perception, in which a model must classify known categories while flagging unseen samples as unknown. Following our midterm proposal, we implement a Newness Transformer with three components: (1) query-level objectness scoring, (2) known-class classification, and (3) a fused newness score derived from energy and prototype distance. Experiments on the UC Berkeley Mill tool-wear dataset use classes 0 and 1 for training and hold out class 2 as the unknown. In our final run, the model achieves a **known recall of 0.8033** and an **unknown precision of 0.4360**, satisfying the project acceptance targets of known recall > 0.80 and unknown precision ≥ 0.35 . We additionally report runtime efficiency and a detailed error analysis.

1 Introduction

1.1 General overview of task and method

Closed-world classifiers assume that all test categories were observed during training. In open-world settings, this assumption fails: unknown samples are frequently misclassified with high confidence or silently absorbed into the background class. Our goal is to preserve known-class accuracy while explicitly detecting novelty at test time. Consistent with our midterm report, we decouple prediction into three sub-tasks: (i) **objectness estimation**, which determines whether a query corresponds to meaningful foreground behavior; (ii) **known-class identity prediction**, which predicts a label among seen classes only; and (iii) **newness scoring**, which rejects unreliable known predictions and emits an unknown label when warranted.

1.2 Past approaches and how ours differs

Open-set and open-world recognition has been approached from several directions. OpenMax [3] models deep-activation tails to estimate whether a sample belongs to an unseen class. Energy-based OOD detection [2] computes a scalar energy value from class logits, exploiting the fact that in-distribution inputs generally produce lower energy than OOD inputs. Open World Object Detection (OWOD) [1] extends detection pipelines with an unknown-aware proposal mechanism and an incremental learning protocol.

These methods emphasize different signals—activation geometry, uncertainty, or proposal structure. Our model combines uncertainty from class logits with prototype-distance geometry in one pipeline, then tunes the operating threshold on validation data to satisfy the target known-recall and unknown-precision criteria rather than AUROC alone.

2 Methodology

2.1 Model architecture

The implementation of our model follows our midterm design and has three main modules.

Transformer backbone (`src/models/transformer.py`). Sensor windows are split into 1-D patches, embedded, and processed by an encoder-decoder transformer with DETR-style cross-attention [4]. Multiple learnable queries let one query capture foreground evidence while the others act as background or surrogate-unknown alternatives.

Prototype-Attention Memory (`src/models/pam.py`). PAM maintains one prototype per known class using exponential moving-average updates. At inference, the minimum query-to-prototype distance $D(q_j)$ provides a geometric novelty signal: small values indicate known-like features, while large values suggest novelty.

Newness scoring and prediction (`src/models/newness_model.py`). Given class logits $\mathbf{z} \in \mathbb{R}^K$, the energy score is

$$E(q_j) = -\log \sum_k \exp(z_k).$$

Both energy and prototype distance are normalized and fused as

$$S_{\text{new}}(q_j) = \alpha \cdot \hat{E}(q_j) + (1 - \alpha) \cdot \hat{D}(q_j),$$

where α is the `fusion_alpha` in `configs/default.yaml` that was adjusted by us during training. The final open-world decision flags a sample as unknown when $S_{\text{new}} > \tau$, with τ tuned on validation data.

2.2 Training objective

The loss in `src/losses/losses.py` combines four terms after Hungarian matching:

1. **Classification cross-entropy** for known classes, with class balancing.
2. **Objectness BCE** for matched foreground versus unmatched queries.
3. **Energy margin/rank loss** to separate in-distribution and OOD energy.
4. **Prototype contrastive loss** to tighten within-class features and separate prototypes.

2.3 Data protocol and preprocessing

We train on known classes $\{0, 1\}$ and hold out class $\{2\}$ as unknown. The final split sizes are train 14994, validation 4053, and test 4053 respectively. Unknown labels are remapped to -1 during open-world evaluation, and normalization is fit only on training-known samples to avoid leakage of data in our study design.

2.4 Experimental journey

During our training process, we had seven notable iterations of our model, each targeting one major bottleneck.

Iteration	Main change	Practical effect
I1	Built initial backbone, PAM path, and train/eval pipeline	Established the end-to-end baseline
I2	Revised loss path and added stronger classification/objectness controls	Improved optimization stability
I3	Refactored loss weights; added class balancing and energy-penalty controls	Reduced coupling between known retention and unknown separation
I4	Tuned default energy/loss weights and added quick-preview config	Faster ablation cycles, fewer wasted long runs
I5	Aligned checkpoint scoring and threshold selection with acceptance criteria	Model selection became target-driven (known recall , unknown precision)
I6	Optimized open-world behavior; added EMA evaluation and rank-margin energy training	Stronger deployment-oriented robustness and checkpoint reliability
I7	Added ensemble/selection utilities, Longleaf submission script, and final run artifact	Completed reproducible final workflow and report traceability to our final run

Overall, the final result came from iterative refinement of optimization, thresholding, and reproducibility rather than a single architectural change, and each member in our team all contributed to certain iterations.

3 Experiments

3.1 Training details

All reported metrics come from our final run, `result/COMP560-oncp-44468619.out`, which was launched by `longleaf_train.pbs` on Longleaf Cluster with `configs/default.yaml`. The final run used one V100 GPU, 8 CPU cores, and 32 GB RAM under PyTorch 2.5.1+cu121. The 1.23M-parameter model was trained with AdamW, cosine decay, warmup, and batch size 128.

3.2 Results and analysis

Final test metrics are reported in Table 1. Both acceptance criteria are satisfied: known recall > 0.80 and unknown precision ≥ 0.35 .

Performance analysis. Binary accuracy is high (0.8919), known recall clears the target at 0.8033, and unknown precision reaches 0.4360, also above target. Energy-based scoring (0.6869 AUROC) is

Table 1: Final test metrics.

Metric	Value
Known recall	0.8033
Unknown precision	0.4360
Unknown recall	0.2310
Binary accuracy	0.8919
AUROC (newness)	0.7059
AUPR (newness)	0.3709
AUROC (energy)	0.6869
AUROC (distance)	0.5666
Threshold mode	<code>val_tuned:meets_known_and_precision</code>

stronger than prototype distance (0.5666), while their fusion performs best at 0.7059, showing that the signals are complementary.

Error analysis. The main weakness is low unknown recall (0.2310): many true unknowns are still assigned to known classes. This reflects conservative threshold tuning and likely overlap between class 2 and the known classes in sensor-feature space.

3.3 Efficiency analysis

Training was relatively efficient for us: early stopping ends training after 267 seconds (4.45 minutes), final reporting finishes in 274 seconds, and average epoch time is about 7.2 seconds. That corresponds to roughly 0.074 GPU-hours per run and about 2078 samples/s. Efficiency comes mainly from early stopping, EMA-based checkpoint comparison, and acceptance-aware threshold selection.

4 Conclusion and Discussion

The final implementation remains consistent with our midterm direction and satisfies both project acceptance targets on our final run. By combining objectness scoring, energy-based uncertainty, and prototype geometry within a single transformer pipeline, the model achieves effective open-world behavior on the Mill split while keeping computation modest.

The main limitation is low unknown recall (0.2310), reflecting a conservative boundary that under-detects novel inputs. Relaxing τ could recover more unknowns, but at the risk of violating the known-recall target. The evaluation also uses only one held-out unknown class, which limits generality. Future work should improve unknown recall at fixed precision, strengthen calibration or outlier exposure [5], and test additional held-out unknown classes and domain shifts.

References

- [1] K. J. Joseph, S. Khan, F. S. Khan, and F. M. Zampetti, “Towards Open World Object Detection,” *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [2] W. Liu, X. Wang, J. Owens, and Y. Li, “Energy-Based Out-of-Distribution Detection,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [3] A. Bendale and T. Boulton, “Towards Open Set Deep Networks,” *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [4] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-End Object Detection with Transformers,” *Proc. European Conference on Computer Vision (ECCV)*, 2020.
- [5] D. Hendrycks, M. Mazeika, and T. Dietterich, “Deep Anomaly Detection with Outlier Exposure,” *Proc. International Conference on Learning Representations (ICLR)*, 2019.